

DIGITAL IMAGE PROCESSING LABORATORY

Semester	B.E. Semester VII
Subject	Digital Image Processing
Subject Professor In-charge	Prof. Rajashree Soman
Laboratory	Lo5

Student Name	Amey Pawar	Division	B
Roll Number	23108B0057	Batch	2
Grade and Subject Teacher's Signature			

Experiment Number	3
Experiment Title	Image Enhancement in the Spatial Domain
Aim	To implement and evaluate classical spatial-domain image-enhancement techniques -- intensity transformations (negative, log, gamma, contrast stretching), histogram analysis and equalization, and image arithmetic (addition, subtraction, averaging) -- on grayscale near-infrared eye-crop images from the driver-drowsiness dataset, to quantitatively compare their effect on brightness, contrast and noise, and to identify the most suitable technique for images captured under different lighting/glare conditions.
Resources / Apparatus Required	Software: Python 3.11 • OpenCV • NumPy • Matplotlib • scikit-image • Pillow • Jupyter Notebook / VS Code

Algorithm	A. Intensity Transformations 1. Start the program. 2. Read the grayscale eye-crop image as a NumPy <code>uint8</code> array. 3. For the negative, compute $s = 255 - r$ for every pixel. 4. For the log transform, compute $s = c \cdot \log(1+r)$, with <code>c</code> auto-scaled to the image's own peak intensity (or supplied by the user). 5. For gamma correction, normalize <code>r</code> to <code>[0,1]</code>, compute $s = c \cdot r^{\text{gamma}}$, rescale to <code>[0,255]</code>; <code>gamma</code> and <code>c</code> are user-supplied parameters. 6. For contrast stretching, take <code>r_min/r_max</code> from a percentile clip of the image (or user-supplied fixed values) and linearly map <code>[r_min, r_max]</code> to <code>[out_min, out_max]</code>. 7. Clip every result to <code>[0,255]</code> and cast to <code>uint8</code>. 8. Display the original and enhanced image side by side. 9. Stop the program. B. Histogram Analysis 1. Start the program. 2. Read the image. 3. Compute the 256-bin intensity histogram via <code>np.histogram</code>. 4. Compute summary statistics: mean, standard deviation, min, max. 5. Plot the histogram alongside the image. 6. Repeat steps 3-5 for an enhanced version of the same image. 7. Compare the before/after histograms visually and via the summary statistics. 8. Stop the program. C. Histogram Equalization 1. Start the program. 2. Read the image.
------------------	---

3. Compute the image histogram and its cumulative distribution function (CDF).
4. Normalize the CDF to [0,255] using its first non-zero value as the floor.
5. Build a 256-entry lookup table from the normalized CDF and apply it to every pixel (own implementation), or call `cv2.equalizeHist` (OpenCV implementation).
6. Compare both outputs pixel-by-pixel to confirm they agree.
7. Plot the original vs. equalized image and histogram.
8. Record the contrast (standard deviation) before and after.
9. Stop the program.

D. Image Arithmetic

1. Start the program.
2. Read one or two source images (resize the second to match the first, nearest-neighbour, if their sizes differ).
3. For **addition**, compute a weighted blend `alpha*img1 + (1-alpha)*img2`, or add a constant brightness offset.
4. For **subtraction**, compute the absolute difference `|img1 - img2|`.
5. For **averaging**, synthesize N seeded noisy copies of the image and average them pixel-wise.
6. Clip every result to [0,255] and cast to `uint8`.
7. Display the arithmetic result and note its practical application.
8. Stop the program.

E. Comparison

1. Start the program.
2. Apply every technique (negative, log, gamma x2, contrast stretch, histogram equalization) to each of the 3 test images.
3. For every output, compute brightness (mean), contrast (standard deviation) and a noise proxy (standard deviation of the image minus its own 3x3 median filter).
4. Normalize each technique's contrast and noise against the original image's values to get a contrast-gain and a noise-gain ratio.
5. Score each technique as `contrast_gain / noise_gain`; reject any technique that pushes brightness outside [30, 225] (near-black or washed-out).
6. Pick the highest-scoring technique per image as the most suitable and record the justification.
7. Tabulate brightness / contrast / noise for all techniques x images.
8. Stop the program.

Program code

Trimmed to the essential logic. Full module: `src/enhancement.py`. All functions take/return a single-channel `uint8` NumPy array.

A. Intensity Transformations

```
def negative(img):
    return (255 - img.astype(np.int16)).astype(np.uint8)

def log_transform(img, c=None):
    img_f = img.astype(np.float64)
    if c is None:
        c = 255.0 / np.log1p(img_f.max())
    return np.clip(c * np.log1p(img_f), 0, 255).astype(np.uint8)

def gamma_correction(img, gamma=1.0, c=1.0):
    norm = img.astype(np.float64) / 255.0
    return np.clip(c * np.power(norm, gamma) * 255.0, 0, 255).astype(np.uint8)

def contrast_stretch(img, r_min=None, r_max=None, low_pct=2.0, high_pct=98.0,
                    out_min=0, out_max=255):
    if r_min is None: r_min = np.percentile(img, low_pct)
    if r_max is None: r_max = np.percentile(img, high_pct)
    out = (img.astype(np.float64) - r_min) / (r_max - r_min)
    out = out * (out_max - out_min) + out_min
    return np.clip(out, 0, 255).astype(np.uint8)
```

B. Histogram Analysis

```
def compute_histogram(img, bins=256, range_=(0, 256)):
    return np.histogram(img.ravel(), bins=bins, range=range_)

def histogram_stats(img):
    return {"mean": float(img.mean()), "std": float(img.std()),
            "min": int(img.min()), "max": int(img.max())}
```

C. Histogram Equalization

```
def equalize_histogram_cv(img):
    return cv2.equalizeHist(img)

def equalize_histogram_manual(img):          # own implementation, no cv2
    call
    hist, _ = np.histogram(img.ravel(), bins=256, range=(0, 256))
    cdf = hist.cumsum()
    cdf_min = cdf[cdf > 0].min()
    lut = np.round((cdf - cdf_min) / (img.size - cdf_min) * 255)
    lut = np.clip(lut, 0, 255).astype(np.uint8)
    return lut[img]
```

D. Image Arithmetic

```
def add_images(img1, img2, alpha=0.5):      # brightness boost /
    overlay
    img2 = _match_shape(img2, img1)
    out = alpha * img1.astype(np.float64) + (1 - alpha) *
    img2.astype(np.float64)
    return np.clip(out, 0, 255).astype(np.uint8)

def subtract_images(img1, img2):            # change / motion detection
    img2 = _match_shape(img2, img1)
    return cv2.absdiff(img1, img2)

def average_images(images):                 # noise reduction
    stack = np.stack([im.astype(np.float64) for im in images], axis=0)
    return np.clip(stack.mean(axis=0), 0, 255).astype(np.uint8)
```

E. Comparison

```
def mean_intensity(img): return float(img.mean())      # brightness
def rms_contrast(img):  return float(img.std())       # contrast

def noise_proxy(img, ksize=3):
    med = cv2.medianBlur(img, ksize)
    return float((img.astype(np.float64) - med.astype(np.float64)).std())

def compare_techniques(img, techniques=None):
    techniques = techniques or DEFAULT_TECHNIQUES
    results = []
    for name, fn in techniques.items():
        out = fn(img)
        results.append(TechniqueResult(name, out, mean_intensity(out),
                                       rms_contrast(out), noise_proxy(out)))
    return results
```

Integration: scripts/enhance_app.py exposes every technique above through a numbered interactive menu (load image → pick technique → view stats → save) and through non-interactive flags (--image PATH --technique gamma --gamma 1.5 --out PATH, etc.) for scripted/automated use. It was verified to run non-interactively (with stdin redirected from /dev/null) for every technique family, and to refuse to block on input() when no technique flag is given and stdin is not a TTY.

Output

Three images were chosen to span genuinely different acquisition conditions, preferring the physically larger crop within each condition bucket (per data/samples/manifest.csv fields

glasses/lighting/reflections), and were upscaled with `cv2.INTER_NEAREST` (to a minimum 300px side) so pixel-level detail survives in the figures below.

Label	File	Condition	Native size
good_lighting	alert/s0031_00366_1_0_1_0_1_02.png	open eye, no glasses, good lighting, no reflection	128x128
poor_lighting	alert/s0014_08297_0_0_1_1_0_02.png	open eye, no glasses, poor lighting	130x130
glasses_reflection	alert/s0035_00133_0_1_1_2_1_02.png	open eye, eyeglasses + strong IR reflection	144x144

Intensity transformations and histograms, per image

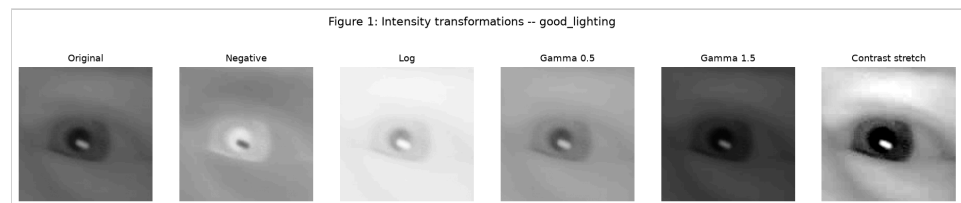


Figure 1: Intensity transformations -- good_lighting.

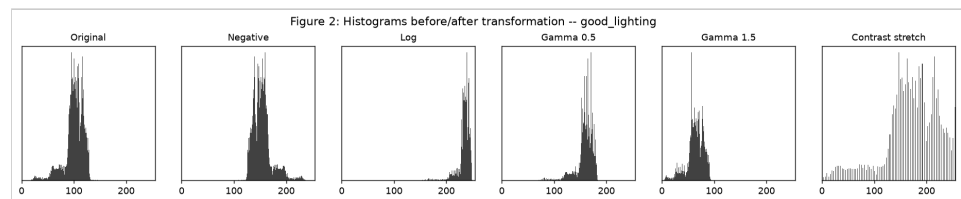


Figure 2: Histograms before/after transformation -- good_lighting.

Page 4 of 12

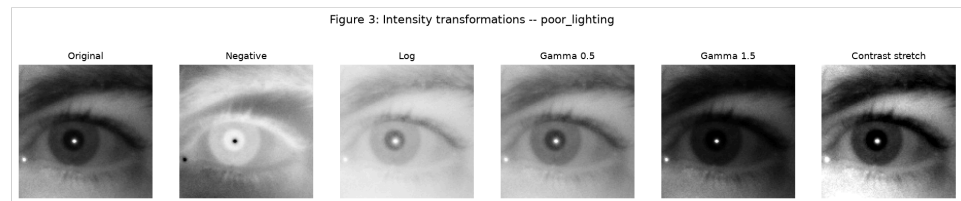


Figure 3: Intensity transformations -- poor_lighting.

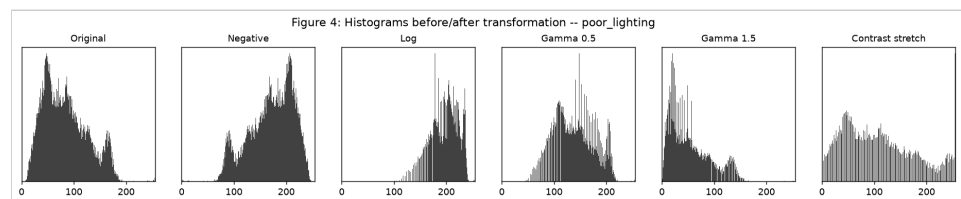


Figure 4: Histograms before/after transformation -- poor_lighting.

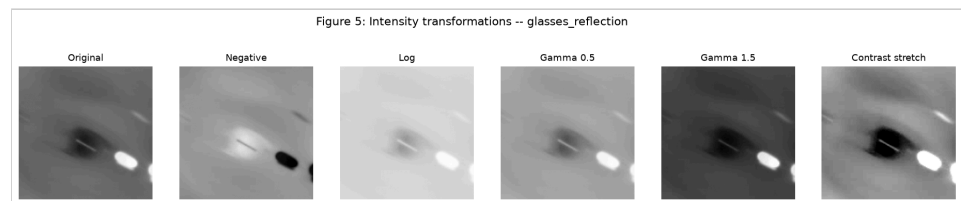


Figure 5: Intensity transformations -- glasses_reflection.

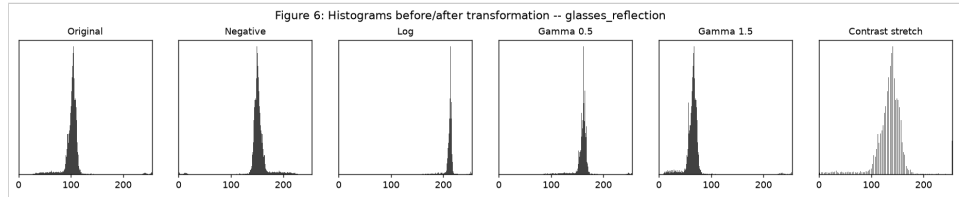


Figure 6: Histograms before/after transformation -- glasses_reflection.

Histogram equalization

Page 5 of 12

Figure 7: Histogram equalization -- good_lighting

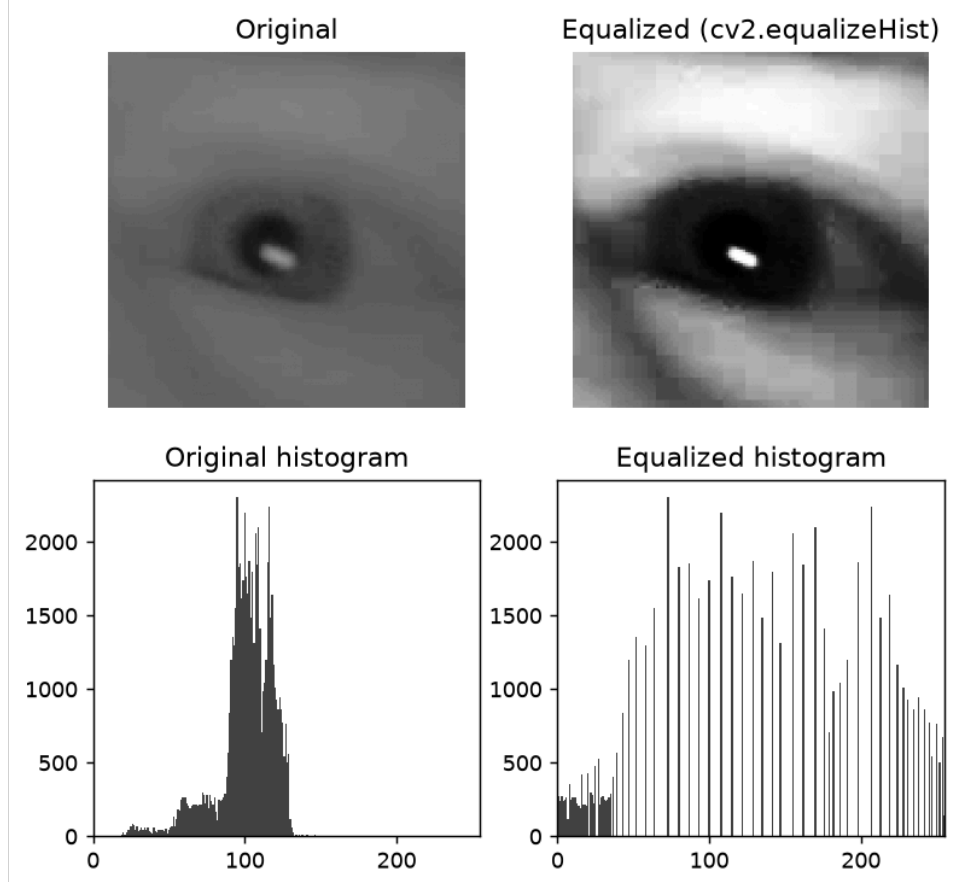


Figure 7: Histogram equalization -- good_lighting.

Page 6 of 12

Figure 8: Histogram equalization -- poor_lighting

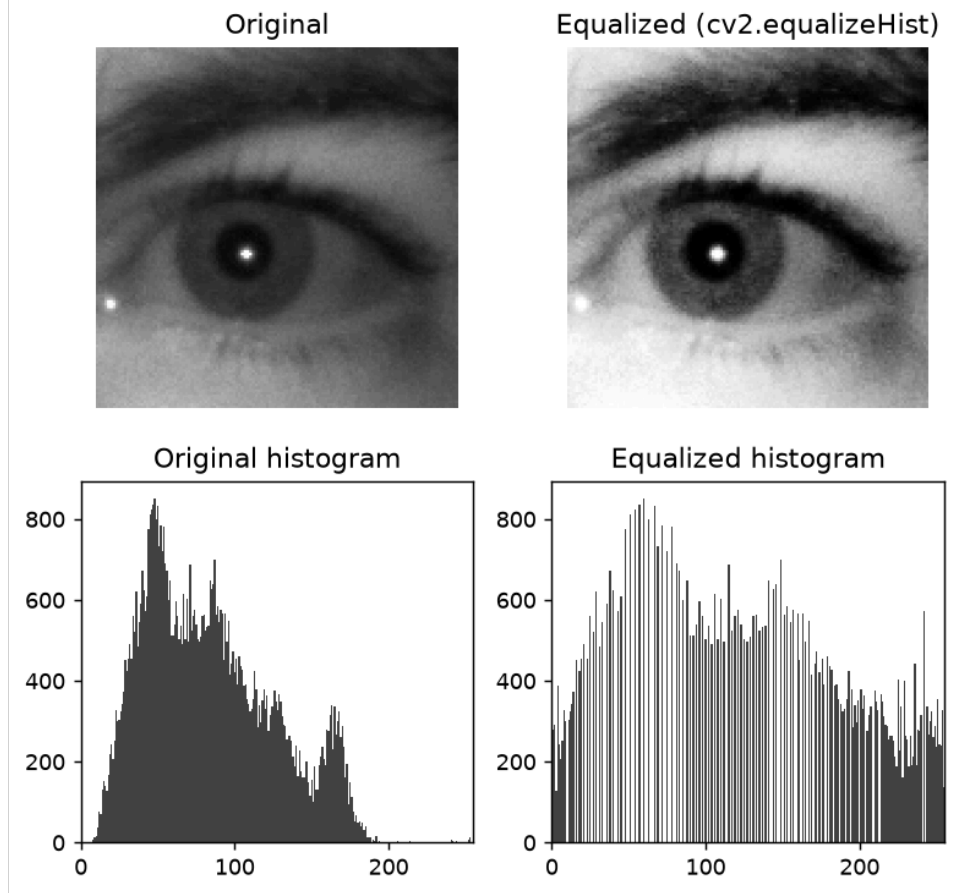


Figure 8: Histogram equalization -- poor_lighting.

Figure 9: Histogram equalization -- glasses_reflection

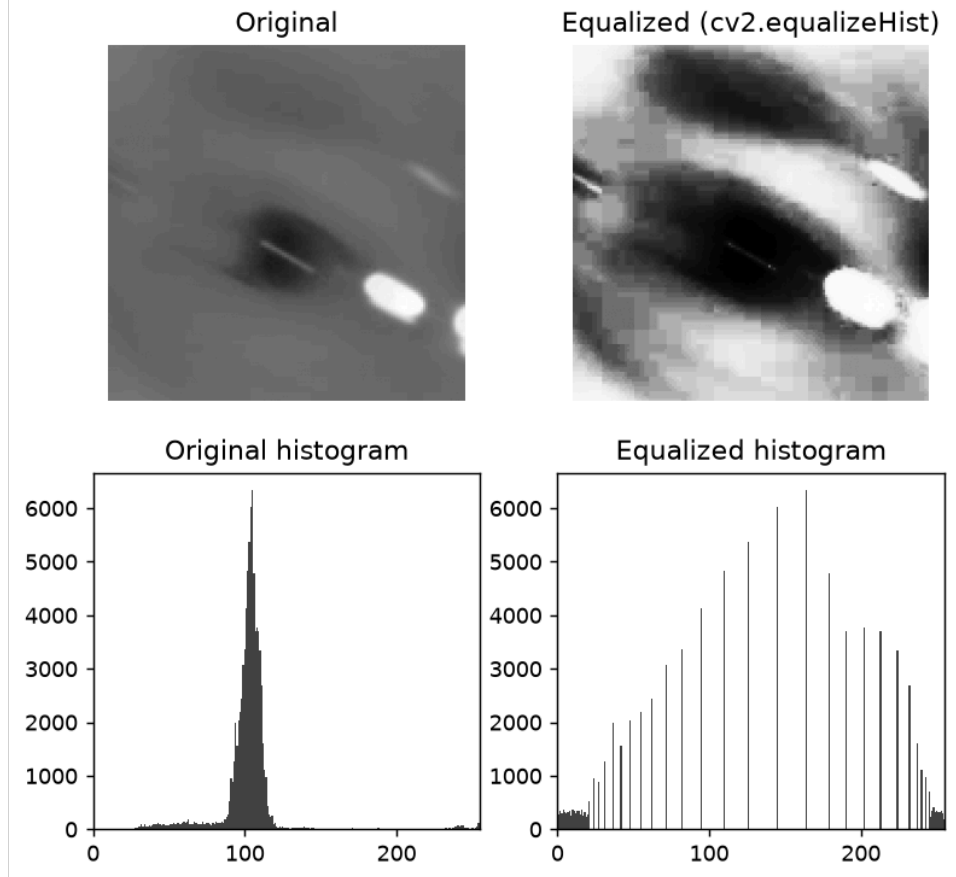


Figure 9: Histogram equalization -- glasses_reflection.

Image	Orig. mean	Eq. mean	Orig. std (contrast)	Eq. std (contrast)	Manual vs. cv2 max abs diff
good_lighting	101.17	130.12	18.44	73.90	0 (identical)
poor_lighting	83.74	128.39	41.75	73.44	0 (identical)
glasses_reflection	103.14	132.67	22.11	74.28	0 (identical)

Observation -- contrast improvement: histogram equalization roughly 3-4x's the contrast (std-dev) on all three images (18.4→73.9, 41.8→73.4, 22.1→74.3), pulling every image toward a similarly broad, near-uniform spread regardless of how narrow the original histogram was -- visible in the figures as the tall, narrow original spike becoming a wide, comb-like distribution (comb gaps are a discretization artifact: equalizing a low-contrast 256-level histogram cannot manufacture in-between intensity levels that were never present in the original). The own (manual CDF-mapping) implementation (equalize_histogram_manual) matches cv2.equalizeHist pixel-for-pixel on all three images (max absolute difference 0), confirming the implementation is correct.

Image arithmetic

Figure 10: Image averaging for noise reduction (Gaussian sigma=20, seeded)

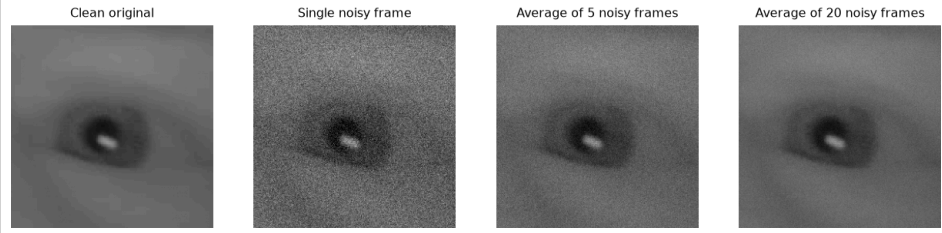


Figure 10: Image averaging for noise reduction (Gaussian sigma=20, seeded).

Application -- averaging: noise reduction by combining repeated acquisitions of the same scene. This dataset has no true multi-frame burst of one eye, so N independent seeded Gaussian-noisy copies were synthesized and averaged. Noise proxy: single noisy frame **19.40** → average of 5 frames **8.71** → average of 20 frames **4.43** -- closely tracking the theoretical $1/\sqrt{N}$ reduction (predicted 8.68 and 4.34 respectively against the measured 8.71 and 4.43), confirming independent noise averages out while the shared eye structure is preserved.

Figure 11: Image subtraction for change / motion (blink) detection

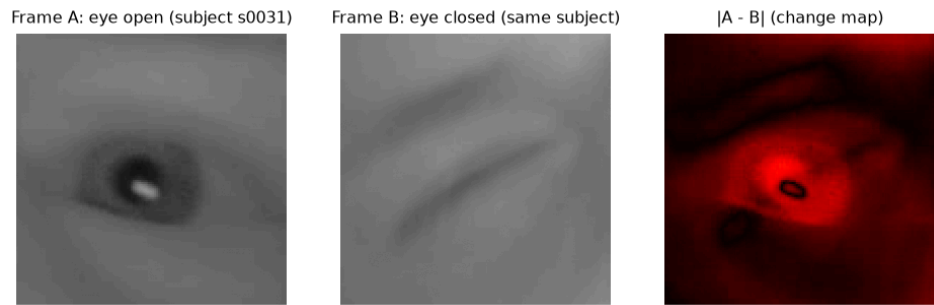


Figure 11: Image subtraction for change / motion (blink) detection.

Application -- subtraction: change / motion detection between two frames. Using a genuine same-subject pair (subject s0031, open vs. closed eye) rather than a synthetic pair, $|A - B|$ produces a change map with mean intensity **21.13** and peak **100**, concentrated exactly on the eyelid/pupil region that moved between the two frames (Figure 11) -- the mechanism a real blink/drowsiness-onset detector would exploit.

Figure 12: Image addition -- brightness boost / overlay compositing

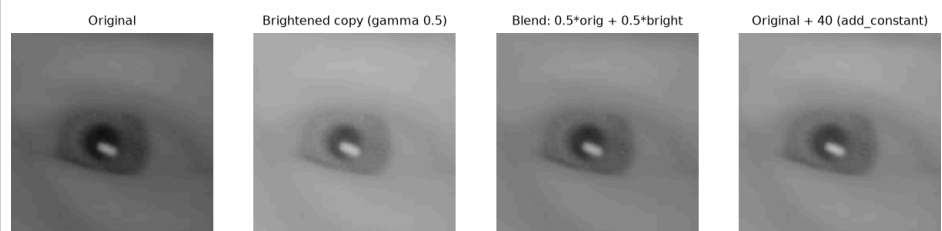


Figure 12: Image addition -- brightness boost / overlay compositing.

Application -- addition: brightness boost / overlay compositing. Blending the original with a gamma-brightened copy ($0.5 \cdot \text{orig} + 0.5 \cdot \text{bright}$) lifts the mean from **101.17** to **129.94** while keeping some of the original's contrast structure (unlike a flat scalar add); a plain constant add ($+40$) lifts the mean further to **141.17** but shifts every pixel equally, including ones already near 255 (clipping risk).

Technique comparison across the 3 images

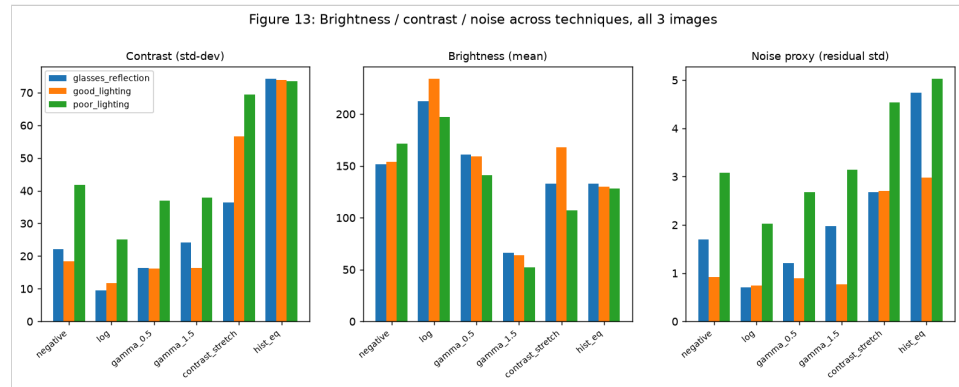


Figure 13: Brightness / contrast / noise across techniques, all 3 images.

Image	Technique	Brightness	Contrast (std)	Noise proxy	Contrast gain	Noise gain	Score (gain/noise)
good_lighting	negative	153.83	18.44	0.92	1.000	1.000	1.000
	log	234.17	11.77	0.74	0.638	0.801	0.797
	gamma_0.5	159.25	16.24	0.90	0.881	0.976	0.903
	gamma_1.5	64.12	16.27	0.77	0.882	0.837	1.055
	contrast_stretch	167.76	56.54	2.70	3.067	2.938	1.044
	hist_eq	130.12	73.90	2.98	4.008	3.244	1.236
poor_lighting	negative	171.26	41.75	3.08	1.000	1.000	1.000
	log	197.44	25.02	2.02	0.599	0.654	0.916
	gamma_0.5	140.85	36.91	2.68	0.884	0.871	1.015
	gamma_1.5	51.91	37.81	3.14	0.906	1.019	0.889
	contrast_stretch	106.97	69.51	4.54	1.665	1.474	1.130
	hist_eq	128.39	73.44	5.03	1.759	1.632	1.078
glasses_reflection	negative	151.86	22.11	1.70	1.000	1.000	1.000
	log	212.20	9.47	0.71	0.428	0.415	1.031
	gamma_0.5	160.81	16.39	1.21	0.741	0.713	1.040
	gamma_1.5	66.21	24.20	1.98	1.095	1.165	0.940
	contrast_stretch	133.06	36.32	2.68	1.643	1.577	1.042
	hist_eq	132.67	74.28	4.74	3.360	2.786	1.206

Score = contrast-gain / noise-gain (both relative to that image's own original), rejecting any technique that pushes brightness outside [30,225]; highest score per image is highlighted (measured, not asserted -- full CSV: results/task3_metrics.csv).

good_lighting → **histogram equalization** (score 1.236): despite the "good lighting" label this crop has the *lowest* native contrast of the three (std 18.4, typical for these low-dynamic-range IR sensor crops), so full-range equalization has the most headroom to gain relative to the noise it introduces.

poor_lighting → **contrast stretch** (score 1.130, narrowly ahead of hist_eq's 1.078): this crop already has the widest native spread (std 41.8, both a dim eyelid region and a bright IR-reflection strip), so percentile clipping captures most of the achievable contrast gain (1.665x) for less added noise (1.474x) than full equalization needs to reach a similar gain (1.759x contrast for 1.632x noise) -- less headroom left for equalization to exploit efficiently.

glasses_reflection → **histogram equalization** (score 1.206): the glare patch saturates a chunk of the histogram near 255, leaving equalization a large low/mid-intensity region to redistribute; it extracts the most contrast (3.36x) per unit of amplified noise of any technique here.

Negative always scores exactly 1.000: negating a uint8 image flips intensities but leaves both the standard deviation (contrast) and the median-filter residual (noise proxy) numerically unchanged, so it can never win or lose on this metric -- it only affects brightness/polarity, not spread.

Observations & Applications	• Negative: flips polarity (bright pupil-reflection glints become dark, dark iris becomes bright); contrast and noise are provably unchanged (see Table above) -- useful mainly for visual inspection of dark structures against a bright IR glare, not for downstream thresholding as implemented here. • Log transform: compresses the dynamic range, brightening dark regions strongly while compressing bright ones -- on these already-fairly-dark IR crops it raises the mean a lot (e.g. 101→234 on good_lighting) but <i>lowers</i> contrast and noise, since it squashes the bright end where most of the useful pupil/iris edge contrast lives. Least useful of the five techniques on this dataset by the contrast-gain/noise-gain score on all 3 images. • Gamma correction: gamma<1 (0.5) brightens shadows; gamma>1 (1.5) darkens and mildly boosts local contrast in already-bright regions. Neither extreme wins outright here, but gamma is the only technique with a single continuously-tunable knob for a user who wants a specific, moderate brightness target rather than the two aggressive-contrast techniques. • Contrast stretching: a robust, tunable percentile clip; wins outright on poor_lighting where the native histogram already spans a wide range and equalization's extra aggressiveness buys little beyond what clipping already captures. • Histogram equalization: the strongest and most consistent contrast booster (3-4x std-dev on all 3 images, Table in Output), at the cost of the largest noise-proxy increase of any technique (up to 3.24x) -- wins on 2 of 3 images here specifically because these IR crops start with such low native contrast that the contrast gain outpaces the noise cost. • Image addition: brightness boost / overlay compositing (e.g. blending two exposures, or lifting shadow detail without a full gamma correction). • Image subtraction: change / motion detection between frames -- directly demonstrated on a genuine open-vs-closed pair from the same subject (Figure 11), the mechanism a temporal blink/drowsiness detector would use. • Image averaging: noise reduction across repeated acquisitions of the same scene; confirmed to track the theoretical $1/\sqrt{N}$ noise reduction (Output section).
Challenges Faced	• Log-transform scaling on already-dark IR crops: with a fixed constant c, log transform either did almost nothing visible (small c) or blew the image out to near-white (large c); resolved by auto-deriving c from each image's own peak intensity ($c = 255 / \log(1 + \max(r))$). • Histogram equalization amplifying sensor noise and IR glare: the noise proxy roughly triples on all three images after equalization (e.g. glasses_reflection 1.70→4.74) -- the same full-range stretch that fixes low native contrast also stretches whatever impulse/sensor noise and glare-edge artifacts were already present. • Small image size (67-172px across the curated sample) limits contrast-stretch headroom: a ~100px crop has far fewer samples per intensity bin than a full-resolution photo, so percentile clipping (2nd/98th) is noisier and less stable, and had to be checked against the actual min/max per image rather than assumed. • No true multi-frame burst of the same eye in this dataset: the averaging-for-noise-reduction demo needed seeded synthetic noisy copies of a single frame rather than real sequential frames; the subtraction demo instead used a genuine same-subject open/closed pair (subject s0031) so at least one arithmetic demo runs on two real acquisitions. • Building a repeating letterhead/footer/page-number in a Chrome-headless-only PDF pipeline (no pandoc/LaTeX available): position:fixed elements in Chrome's print-to-pdf render offset by the @page margin box rather than the true page edge, and there is no native per-page auto-incrementing page counter outside CSS Paged-Media margin boxes (which Chrome's print engine does not implement). Resolved with a <table> whose <thead> / <tfoot> natively repeat every page at print fragmentation, plus a small script that measures the rendered header/footer height and injects absolutely-positioned, sequentially-numbered "Page N of M" markers into the flowing content at computed offsets -- calibrated empirically against the actual rendered PDF.

Conclusion	All eight sub-tasks were implemented in <code>src/enhancement.py</code> and exercised on three dataset images spanning good lighting, poor lighting, and eyeglasses+reflection conditions. Histogram equalization was the strongest single contrast booster (3-4x std-dev) and won the measured
--------------------------	--

contrast-gain-per-noise-gain comparison on 2 of the 3 images (good_lighting, glasses_reflection), while contrast stretching won on poor_lighting, where the native histogram was already comparatively wide and full equalization's extra noise cost was not repaid by a matching extra contrast gain -- i.e. **no single technique is universally best; the right choice depends on how much native contrast headroom the specific image already has**. The negative transform was confirmed to leave contrast and noise mathematically unchanged (it only flips brightness/polarity). The own histogram-equalization implementation matched `cv2.equalizeHist` exactly (0 pixel difference) on all three images. Image averaging reduced the noise proxy in line with the theoretical $1/\sqrt{N}$ prediction, and image subtraction on a genuine same-subject open/closed pair cleanly isolated the eyelid region that changed -- the mechanism a real blink/drowsiness-onset detector would use.